Linear Classification

Muchang Bahng

Spring 2025

Contents

Theorem 0.1 (Minimality of Bayes Classifier) . . . 3
Definition 0.1 (Plug-In Classifier) . . . 4
Theorem 0.2 (Classification Risk Bounded by Regression Risk) . . . 4

1 Perceptron **6**
Definition 1.1 (Perceptron) . . . 6
Definition 1.2 (Surrogate Loss for Perceptron) . . . 6

2 Logistic Regression **8**
Definition 2.1 (Logistic Regression) . . . 8
Theorem 2.1 (Decision Boundary is Linear) . . . 8
2.1 Maximum Likelihood Estimation . . . 8
Lemma 2.1 (Likelihood for Bernoulli) . . . 9
Definition 2.2 (Binary Cross Entropy Loss as Surrogate Loss) . . . 9
Theorem 2.2 (Risk) . . . 9
Example 2.1 (SGD from Scratch) . . . 9
2.2 Significance Tests and Confidence Sets . . . 10
2.3 Concentration Bounds . . . 10

3 Softmax Regression **11**
Definition 3.1 (Softmax) . . . 11
Lemma 3.1 (Derivative of Softmax) . . . 11
Definition 3.2 (One-Hot Encoding) . . . 12
Definition 3.3 (Softmax Regression Model) . . . 12
3.1 Maximum Likelihood Estimation . . . 12
Lemma 3.2 (Likelihood for Multinomial) . . . 12
Definition 3.4 (Multiclass Cross Entropy Loss as Surrogate Loss) . . . 13
Theorem 3.1 (Risk) . . . 13
Theorem 3.2 (Gradient of Softmax Loss) . . . 13
Example 3.1 (SGD from Scratch) . . . 15
3.2 Significance Tests and Confidence Sets . . . 16
3.3 Concentration Bounds . . . 16

4 Regularized Softmax Regression **17**
4.1 Ridge . . . 17
Definition 4.1 (Loss) . . . 17
4.2 Lasso . . . 17

5 Gaussian Discriminant Analysis **18**
Definition 5.1 (Linear Discriminant Analysis) . . . 18
Theorem 5.1 (Decision Boundary is Linear) . . . 18
5.1 Comparison to Logistic Regression . . . 18
5.2 Maximum Likelihood Estimate . . . 18
Lemma 5.1 (Likelihood) . . . 18
Theorem 5.2 (MLE) . . . 19
5.3 Quadratic Discriminant Analysis . . . 20
Definition 5.2 (Gaussian Discriminant Analysis) . . . 20

Theorem 5.3 (Maximum Likelihood Estimate) . . . 20
Theorem 5.4 (Bayes Rule for Two Gaussian Classes) . . . 21
Theorem 5.5 (Decision Boundary is Quadratic) . . . 21
5.4 Multiclass GDA . . . 21
Theorem 5.6 (Optimality of the Bayes Classifier) . . . 21
Theorem 5.7 (Multiclass QDA) . . . 22

6 Linear Support Vector Machines 23
Definition 6.1 (Hinge Loss) . . . 23
Definition 6.2 (Binary Support Vector Machine) . . . 23
Theorem 6.1 (Risk) . . . 23
Example 6.1 (SVMs vs Other Classifiers on Linearly Separable Dataset) . . . 23
6.1 Functional and Geometric Margins . . . 24
Definition 6.3 (Geometric margin) . . . 24
Definition 6.4 (Functional Margin) . . . 25
6.2 Analytical Solution . . . 26
6.3 Significance Tests and Confidence Sets . . . 27
6.4 Concentration Bounds . . . 27
6.5 Nonseparable Case . . . 27

7 Generalized Linear Models 28
Example 7.1 (Disease Epidemic) . . . 29
Example 7.2 (Prey Capture Rate) . . . 29
Example 7.3 (Kyphosis Data) . . . 30
7.1 Exponential Family . . . 30
Definition 7.1 (Exponential Family) . . . 31
Example 7.4 (Gaussian) . . . 31
Example 7.5 (Gaussian with Known Variance) . . . 31
Example 7.6 (Bernoulli) . . . 31
Example 7.7 (Binomial with Known Number of Trials) . . . 32
Example 7.8 (Poisson) . . . 32
7.1.1 Canonical Exponential Family . . . 32
Definition 7.2 (Canonical One-Parameter Exponential Family) . . . 32
Example 7.9 (Poisson) . . . 33
Example 7.10 (Gaussian) . . . 33
Example 7.11 (Bernoulli) . . . 33
Theorem 7.1 (Moments) . . . 33
Example 7.12 (Poisson, Normal and Bernoulli as Exponential Families) . . . 34
Theorem 7.2 (Convexity) . . . 34
7.2 Cumulant Generating Function . . . 35
Definition 7.3 (Score) . . . 35
7.3 Link Functions . . . 36
Example 7.13 (Choosing a Link Function) . . . 36
7.3.1 Canonical Link Functions . . . 37
Definition 7.4 (Canonical Link) . . . 37
Example 7.14 (The Bernoulli in Canonical Form) . . . 38
7.4 Likelihood Optimization . . . 38

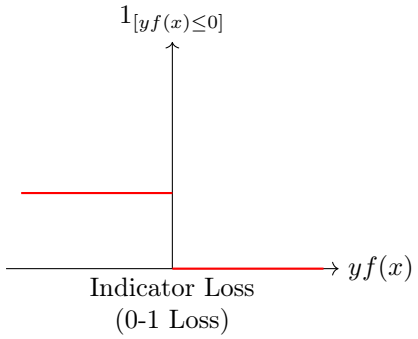
References 39

Now let's talk about linear classification. Remember that the MSE was a nice loss to choose due to the Gauss-Markov theorem, but is there an analogue for classification? Since we are now working in classification, a natural metric would be to see the proportion of samples we get correct, and this is modeled through the 0-1 loss function.

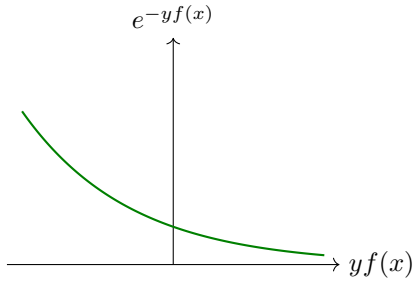
$$L(f, x, y) = \mathbb{1}(y = f(x)) \tag{1}$$

However, this isn't a smooth function to work with, with 0 gradients almost everywhere and hard to solve analytically. We can try a couple things.

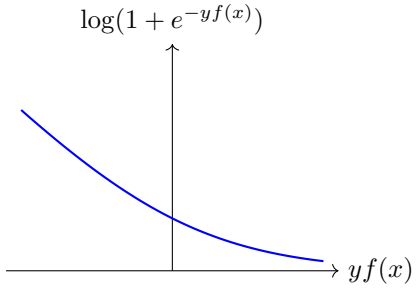
Alternatively, we can use a **surrogate loss function** to approximate the 0-1 loss function. The logistic uses some function, and the SVM uses the smallest convex function to approximate the 0-1 loss function. Here are some examples:



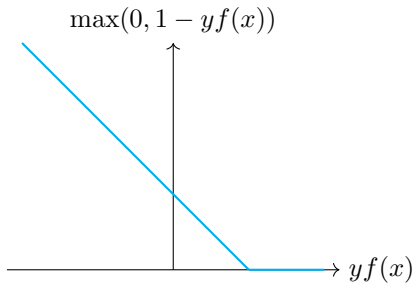
(a) Indicator/0-1 Loss can't be easily optimized.



(b) Exponential Loss for Adaboost.



(c) Log Loss for logistic regression.



(d) Hinge Loss for support vector machines.

Figure 1: Common loss functions used in classification

Intuitively, regression seems like a much harder problem than classification. One predicts over a finite set of values while the other predicts over a continuum. We can try and solve the problem as if we were going regression of values between 0 and 1, and then put a cutoff threshold where it classifies 0 if $f(x) \leq \frac{1}{2}$ and 1 if else. This is called the *Bayes' classifier*.

Theorem 0.1 (Minimality of Bayes Classifier)

The function f^* that minimizes the expected risk of the 0-1 loss is

$$f^*(x) = \begin{cases} 0 & \text{if } g(x) \leq \frac{1}{2} \\ 1 & \text{if } g(x) > \frac{1}{2} \end{cases} \tag{2}$$

where $g(x) = \mathbb{E}[Y \mid X = x] = \mathbb{P}(Y = 1 \mid X = x)$ denotes the regression function.

Proof.

It suffices to show that

$$\mathbb{P}(Y \neq h(X)|X = x) - \mathbb{P}(Y \neq h^*(X)|X = x) \geq 0 \quad \text{for all } x \in \mathcal{X}. \tag{3}$$

Now,

$$\begin{aligned}
\mathbb{P}(Y \neq h(X)|X = x) &= 1 - \mathbb{P}(Y = h(X)|X = x) && (4)\\
&= 1 - (\mathbb{P}(Y = 1, h(X) = 1|X = x) + \mathbb{P}(Y = 0, h(X) = 0|X = x)) && (5)\\
&= 1 - (h(x)\mathbb{P}(Y = 1|X = x) + (1 - h(x))\mathbb{P}(Y = 0|X = x)) && (6)\\
&= 1 - (h(x)m(x) + (1 - h(x))(1 - m(x)))\,. && (7)
\end{aligned}$$

Hence,

$$\begin{aligned}
&\mathbb{P}(Y \neq h(X)|X = x) - \mathbb{P}(Y \neq h^*(X)|X = x) && (8)\\
&= (h^*(x)m(x) + (1 - h^*(x))(1 - m(x))) - (h(x)m(x) + (1 - h(x))(1 - m(x))) && (9)\\
&= (2m(x) - 1)(h^*(x) - h(x)) = 2\left(m(x) - \frac{1}{2}\right)(h^*(x) - h(x)) && (10)
\end{aligned}$$

When $m(x) \geq 1/2$ and $h^*(x) = 1$, (10) is non-negative. When $m(x) < 1/2$ and $h^*(x) = 0$, (10) is again non-negative.

In fact, by fitting a regression function over any data, we can simply compose it with the threshold function to get a binary classifier. This is called the **plug-in classifier**.

Definition 0.1 (Plug-In Classifier)

Given an regression estimator $\hat{g}$, we can define the corresponding **plug-in classifier** as

$$\hat{f}(x) = \begin{cases} 0 & \text{if } \hat{g}(x) \leq \frac{1}{2} \\ 1 & \text{if } \hat{g}(x) > \frac{1}{2} \end{cases} \tag{11}$$

Due to the optimality of the Bayes' classifier, the performance of the plug-in classifier is tied to the performance of the underlying linear regression model. We can actually formalize this with the following.

Theorem 0.2 (Classification Risk Bounded by Regression Risk)

The risk of the plug-in classifier rule in (14) satisfies

$$R(\hat{h}) - R^* \leq 2\sqrt{\int (\hat{m}(x) - m^*(x))^2 dP_X(x)}. \tag{12}$$

Proof.

In the proof of the previous theorem we showed that

$$\begin{aligned}
\mathbb{P}(Y \neq \hat{h}(X)|X = x) - \mathbb{P}(Y \neq h^*(X)|X = x) && (13)\\
= (2\hat{m}(x) - 1)(h^*(x) - \hat{h}(x)) && (14)\\
= |2\hat{m}(x) - 1|I(h^*(x) \neq \hat{h}(x)) && (15)\\
= 2|\hat{m}(x) - 1/2|I(h^*(x) \neq \hat{h}(x)). && (16)
\end{aligned}$$

Now, when $h^*(x) \neq \hat{h}(x)$, there are two possible cases: (i) $\hat{h}(x) = 1$ and $h^*(x) = 0$; (ii) $\hat{h}(x) = 0$ and $h^*(x) = 1$. In both cases, we have that $|\hat{m}(x) - m^*(x)| \geq |\hat{m}(x) - 1/2|$. Therefore,

$$\begin{aligned}
\mathbb{P}(\hat{h}(X) \neq Y) - \mathbb{P}(h^*(X) \neq Y) &= 2 \int |\hat{m}(x) - 1/2| I(h^*(x) \neq \hat{h}(x)) dP_X(x) && (17)\\
&\leq 2 \int |\hat{m}(x) - m^*(x)| I(h^*(x) \neq \hat{h}(x)) dP_X(x) && (18)\\
&\leq 2 \int |\hat{m}(x) - m^*(x)| dP_X(x) && (15)\\
&\leq 2 \sqrt{\int (\hat{m}(x) - m^*(x))^2 dP_X(x)}. && (16)
\end{aligned}$$

The last inequality follows from the fact that $\mathbb{E}|Z| \leq \sqrt{\mathbb{E}Z^2}$ for any Z.

1 Perceptron

The perceptron uses a linear regression function as a plugin-classifier, inspired by the artificial modeling of a human neuron [MP43].

Definition 1.1 (Perceptron)

The **perceptron model** is a discriminative linear classifier that assigns

$$f_w(x) = \begin{cases} 1 & \text{if } w^T x + b \geq 0 \\ -1 & \text{if } w^T x + b < 0 \end{cases} \tag{19}$$

where we have chosen to label class $C_1 = 1$ and $C_2 = -1$.

Note that unlike linear regression (and logistic regression, as we will see later), the perceptron is not a probabilistic model. It is a *discriminant function*, which just gives point estimates of the classes, not their respective probabilities. Like logistic regression, however, it is a linear model, meaning that the decision boundary it creates is always a linear (affine) hyperplane.

To construct the surrogate loss function, we would want a loss that penalizes not only if there is a misclassification, but how *far* that misclassified point is from the boundary. Therefore, if y and $\hat{y} = f_w(x)$ have the same sign, i.e. if $yf_w(x) > 0$, then the penalty should be 0, and if it is < 0, then the penalty should be proportional to the orthogonal distance of the misclassified point to the boundary, which is represented by $-wTxy$ (where the negative sign makes this cost term positive).

Definition 1.2 (Surrogate Loss for Perceptron)

Therefore, our cost functions would take all the points and penalize all the terms by 0 if they are correctly classified and by $-\mathbf{w}^T \boldsymbol{\phi}^{(n)} y^{(n)}$ if incorrectly classified.

$$L(y, \hat{y}) = \sum_{n=1} [-\mathbf{w}^T \boldsymbol{\phi}^{(n)} y^{(n)}]_+ \text{ where } [f(\mathbf{x})]_+ \coloneqq \begin{cases} f(\mathbf{x}) & \text{if } f(\mathbf{x}) > 0 \\ 0 & \text{else} \end{cases} \tag{20}$$

Note that this is a piecewise linear function and convex.

Code 1.1 (Perceptron in scikit-learn)

Let's implement this in scikit-learn, using two pipelines with different data standardization techniques to see the differences in the perceptron boundary.

```python
from sklearn.pipeline import Pipeline
from sklearn.linear_model import Perceptron
from sklearn.preprocessing import QuantileTransformer, StandardScaler

pipe1 = Pipeline([
    ("scale", StandardScaler()),
    ("model", Perceptron())
])

pipe2 = Pipeline([
    ("scale", QuantileTransformer(n_quantiles=100)),
    ("model", Perceptron())
])
```

Figure 2

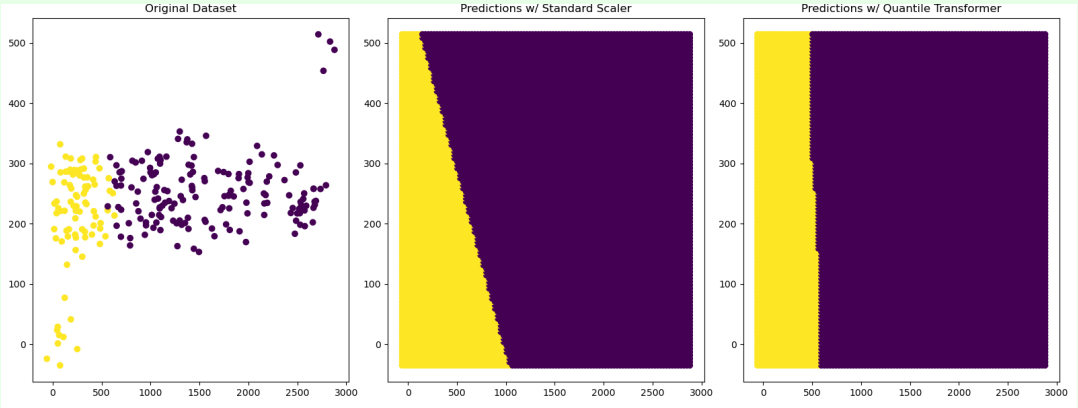


Figure 3: Perceptron Trained on Different Standardized Data

2 Logistic Regression

We can upgrade from a discriminant function to a discriminative probabilistic model with **logistic regression**. In linear regression, we assumed that the targets are linearly dependent with the covariates as $y = w^T x + b$. However, this means that the hypothesis f_w is unbounded. Since we have two classes (say with labels 0 and 1), we must have some sort of *link function* σ that takes the real numbers and compresses it into the domain $[0, 1]$. Technically, we can choose any continuous, monotonically increasing function from $\mathbb{R}$ to $(0, 1)$. However, the following property of the *sigmoid function* $\sigma(x) \coloneqq \frac{1}{1+e^x}$ makes derivation of gradients very nice.

$$\sigma'(x) = \sigma(x)\big(1 - \sigma(x)\big) \tag{21}$$

Definition 2.1 (Logistic Regression)

The **logistic regression** model is probabilistic discriminative classification model

$$Y \mid X = x \sim \text{Bernoulli}(f(x)), \qquad f(x) = \sigma(\beta^T x) \tag{22}$$

where σ is the sigmoid function. It has parameters β.

One important observation to make is that notice that the output of our hypothesis is used as a parameter to define our residual distribution.

1. In linear regression, the f was used as the *mean* μ of a Gaussian.
2. In logistic regression, the f is used also as the mean p of a Bernoulli.

The reason we want this sigmoid is so that we make the domains of the means of the residuals match the range of the outputs of our model. It's simply a manner of convenience, and in fact we could have really chose any function that maps $\mathbb{R}$ to $(0, 1)$.[1]

Note that logistic regression has a *nonlinear regression function*, but a *linear decision boundary*. That is, by running this model through the plugin classifier, this becomes a linear classifier.

Theorem 2.1 (Decision Boundary is Linear)

The decision boundary of logistic regression is linear.

Proof.

We can solve

$$\frac{1}{1 + e^{\beta^T x}} = \frac{1}{2} \implies 2 = 1 + e^{\beta^T x} \implies 1 = e^{\beta^T x} \implies 0 = \beta^T x \tag{23}$$

which defines a linear hyperplane orthogonal to vector β.

2.1 Maximum Likelihood Estimation

Our next job is to define a loss to optimize. The pdf of a Bernoulli is not well-defined, but we can smoothly interpolate between the two points at $x = 0$ and $x = 1$ to get a smooth surrogate likelihood.

[1] Some questions may arise, such as "why isn't the variance parameter of the Gaussian considered in the linear model?" or "what about other residual distributions that have multiple parameters?" This is all answered by generalized linear models, which uses the output of a linear model as a *natural parameter* of the canonical exponential family of residual distributions.

Lemma 2.1 (Likelihood for Bernoulli)

A surrogate likelihood of a Bernoulli(p) random variable is

$$\mathbb{P}(X = x) = p^x(1-p)^{1-x} \tag{24}$$

TBD. Why do we express it in this form? Just for convenience?

Proof.

Just substitute $\mathbb{P}(X = 1) = p^1(1-p)^0 = p$ and $\mathbb{P}(X = 0) = p^0(1-p)^1 = 1-p$.

Therefore, by taking the negative log-likelihood, we can get our loss function.

Definition 2.2 (Binary Cross Entropy Loss as Surrogate Loss)

The surrogate loss for logistic regression is the **binary cross entropy loss**, which is defined as

$$L(y, \hat{y}) = -y \log \hat{y} - (1-y) \log(1-\hat{y}) \tag{25}$$

Now we can define our risk.

Theorem 2.2 (Risk)

The expected risk of logistic regression on the binary cross entropy loss is

$$R(f) = \mathbb{E}_{x,y}\left[-y \log(\sigma(\beta^T x)) - (1-y)\log(1-\sigma(\beta^T x))\right] \tag{26}$$

and the empirical risk on a dataset $\mathcal{D} = \{x^{(i)}, y^{(i)}\}_{i=1}^n$ is

$$\hat{R}(f) = \frac{1}{n}\sum_{i=1}^{n} -y^{(i)} \log(\sigma(\beta^T x^{(i)})) - (1-y^{(i)})\log(1-\sigma(\beta^T x^{(i)})) \tag{27}$$

Unfortunately, there is no closed form solution for logistic regression like the least squares solution in linear regression. Therefore, we can only resort to numerically optimizing it.

Example 2.1 (SGD from Scratch)

Let's implement this for logistic regression.

```
>>> import numpy as np
>>> def sigmoid(z):
...     return 1 / (1 + np.exp(-z))
...
>>>
>>> n, d = 200, 5
>>> beta_true = np.array([1, 2, 3, 1, -2])
>>> X = np.random.randn(n, d)
>>> logits = X.dot(beta_true)
>>> probs = sigmoid(logits)
>>> Y = np.random.binomial(1, probs, n)
>>> lr = 0.5
>>> beta_hat = np.random.randn(5) * 0.1
>>> for _ in range(1000):
...     Y_hat = sigmoid(X.dot(beta_hat))
```

```
...     grad = (1/n) * X.T.dot(Y_hat - Y)
...     beta_hat -= lr * grad
...
>>>
>>> print(beta_hat)
[ 1.18478017  1.92729039  3.20635381  0.65779351 -2.67062206]
```

It turns out that this does not converge as well as linear regression, with more noisy estimates. Usually, we need a larger stepsize and more epochs.

2.2 Significance Tests and Confidence Sets

2.3 Concentration Bounds

3 Softmax Regression

We would like to extend this to the multiclass case. In order to do this, we must start with *multivariate* linear regression and produce another link function o that maps it to the parameter space of a multinomial distribution. It should also be a generalization of the sigmoid.

Definition 3.1 (Softmax)

The **softmax function** is defined

$$o(\mathbf{x}) = \frac{e^{\mathbf{x}}}{\|e^{\mathbf{x}}\|} = \frac{1}{\sum_j e^{x_j}} \begin{pmatrix} e^{x_1} \\ \vdots \\ e^{x_D} \end{pmatrix} \tag{28}$$

This is in fact a generalization of the sigmoid. That is, given softmax for 2 classes, we have

$$o\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \frac{1}{e^{x_1} + e^{x_2}} \begin{pmatrix} e^{x_1} \\ e^{x_2} \end{pmatrix} \tag{29}$$

So, the probability of being in class 1 is

$$\frac{e^{x_1}}{e^{x_1} + e^{x_2}} = \frac{1}{1 + e^{x_2 - x_1}} \tag{30}$$

and the logistic sigmoid is just a special case of the softmax function that avoids using redundant parameters. We actually end up overparameterizing the softmax because the probabilities must add up to one. Another reason to choose the softmax is that the total derivative turns out to simplify our loss, which also parallels to the sigmoid.

Lemma 3.1 (Derivative of Softmax)

The derivative of the softmax is

$$Do(\mathbf{x}) = \text{diag}(o(\mathbf{x})) - o(\mathbf{x}) \otimes o(\mathbf{x}) \tag{31}$$

where $\otimes$ is the outer product. That is, let y_i be the output of the softmax. Then, for the 4×4 softmax function, we have

$$Do(\mathbf{x}) = \begin{pmatrix} y_1(1-y_1) & -y_1y_2 & -y_1y_3 & -y_1y_4 \\ -y_2y_1 & y_2(1-y_2) & -y_2y_3 & -y_2y_4 \\ -y_3y_1 & y_3y_3 & y_3(1-y_3) & -y_3y_4 \\ -y_4y_1 & -y_4y_2 & -y_4y_3 & y_4(1-y_4) \end{pmatrix} \tag{32}$$

Proof.

We will provide a way that allows us not to use quotient rule. Given that we are taking the partial derivative of y_i with respect to x_j, we can use the log of it to get

$$\frac{\partial}{\partial x_j} \log(y_i) = \frac{1}{y_i} \frac{\partial y_i}{\partial x_j} \implies \frac{\partial y_i}{\partial x_j} = y_i \frac{\partial}{\partial x_j} \log(y_i) \tag{33}$$

Now the partial of the log term is

$$\log y_i = \log\left(\frac{e^{x_i}}{\sum_l e^{x_l}} = x_i - \log\left(\sum_l e^{x_l}\right)\right. \tag{34}$$

$$\frac{\partial}{\partial x_j}\log(y_i) = \frac{\partial x_i}{\partial x_j} - \frac{\partial}{\partial x_j}\log\left(\sum_l e^{x_l}\right) \tag{35}$$

$$= 1_{i=j} - \frac{1}{\sum_l e^{x_l}} e^{x_j} \tag{36}$$

and plugging this back in gives

$$\frac{\partial y_i}{\partial x_j} = y_i(1_{i=j} - y_j) \tag{37}$$

A way to encode multiple classes is with one-hot encoding.

Definition 3.2 (One-Hot Encoding)

Given K classes $\{1, \dots, K\}$, the **one-hot encoding** of each class is

$$k \mapsto e_k \in \mathbb{R}^K \tag{38}$$

where e_k is the kth basis vector.

We choose such an encoding since $\|e_k - e_{k'}\|$ is constant for all $k \neq k'$. Therefore, all classes are just as distinct from one another.

Definition 3.3 (Softmax Regression Model)

A **softmax regression** model of K classes is a probabilistic classification model

$$Y \mid X = x \sim \text{Multinomial}(f(x)), f(x) = o(Wx + b) \tag{39}$$

where o is the softmax function. It has the parameters $\theta = \{W \in \mathbb{R}^{k \times d}, b \in \mathbb{R}^k\}$.

Again, we have a linear map followed by some link function (the softmax) which allows us to nonlinearly map our unbounded linear outputs to some domain that can be easily parameterized by a probability distribution.

3.1 Maximum Likelihood Estimation

We do the same steps as that of logistic regression.

Lemma 3.2 (Likelihood for Multinomial)

The surrogate likelihood function for a multinomial is

$$\mathbb{P}(X = j) = \mathbb{P}(X = e_j) = \prod_{k=1}^{K} p_k^{(e_j)_k} \tag{40}$$

where we have one-hot encoded the kth class.

Proof.

All terms in the product will be 1 if $k \neq j$, and so the only term remaining will be $p_j^1 = p_j$.

By taking the negative log-likelihood, we can get our loss function.

Definition 3.4 (Multiclass Cross Entropy Loss as Surrogate Loss)

The surrogate loss for softmax regression is the **multiclass cross entropy loss**, which is defined as

$$L(\theta, x, y) = -\sum_{k=1}^{K} y_k \log \big(f(x)\big)_k \tag{41}$$

Now we define our risk.

Theorem 3.1 (Risk)

The expected risk of softmax regression on the cross entropy loss is

$$R(f) = \mathbb{E}_{x,y}\left[-\sum_{k=1}^{K} y_k \log \big(f(x)\big)_k\right] \tag{42}$$

and the empirical risk on a dataset $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^n$ is

$$\hat{R}(f) = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K} y_k \log \big(f(x)\big)_k \tag{43}$$

Since a closed form solution is not available for logistic regression, it is clearly not available for softmax. Note that since we are taking the derivative of a vector w.r.t. a matrix, we will have to work with higher-order tensors. Fortunately, this reduces down to a cute form that we can still compute with matrices.

Theorem 3.2 (Gradient of Softmax Loss)

The gradient of the empirical risk is

$$\nabla_W \hat{R}(f) = \sum_{i=1}^{n} (f(x^{(i)}) - y^{(i)})(x^{(i)})^T \tag{44}$$

$$\nabla_b \hat{R}(f) = \sum_{i=1}^{n} (f(x^{(i)}) - y^{(i)}) \tag{45}$$

Proof.

Let's write the cross entropy loss function

$$\ell(\theta) = -\sum_{i=1}^{N}\sum_{k=1}^{K} y_k^{(i)} \log \big(h_\theta(x^{(i)})\big)_k = -\sum_{i=1}^{N} y^{(i)} \cdot \log \big(h_\theta(x^{(i)})\big) \tag{46}$$

where $\cdot$ is the dot product. The gradient of this function may seem daunting, but it turns out to be very cute. Let us take a single sample $(x^{(i)}, y^{(i)})$, drop the index i, and write

$$z = Wx + b = z \tag{47}$$

$$\hat{y} = a = o(z) \tag{48}$$

$$L = -y \cdot \log(a) = -\sum_{k=1}^{K} y_k \log(a_k) \tag{49}$$

We must compute

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial a}\frac{\partial a}{\partial z}\frac{\partial z}{\partial \theta} \tag{50}$$

We can compute $\partial L/\partial z$ as such, using our derivations for the softmax derivative above. We compute element wise.

$$\begin{aligned}
\frac{\partial L}{\partial z_j} &= -\sum_{k=1}^{K} y_k \frac{\partial}{\partial z_j}\log(a_k) && (51)\\
&= -\sum_{k=1}^{K} y_k \frac{1}{a_k}\frac{\partial a_k}{\partial z_j} && (52)\\
&= -\sum_{k=1}^{K} \frac{y_k}{a_k} a_k(1_{\{k=j\}} - a_j) && (53)\\
&= -\sum_{k=1}^{K} y_k(1_{\{k=j\}} - a_j) && (54)\\
&= \left(\sum_{k=1}^{K} y_k a_j\right) - y_j && (55)\\
&= a_j\left(\sum_{k=1}^{K} y_k\right) - y_j && (56)\\
&= a_j - y_j && (57)
\end{aligned}$$

and combining these gives

$$\frac{\partial L}{\partial z} = (a-y)^T \tag{58}$$

Now, computing $\partial z/\partial W$ gives us a 3-tensor, which is not ideal to work with. However, let us just compute this with respect to the elements again. We have

$$z_k = \sum_{d=1}^{D} W_{kd}x_d + b_k, \qquad \frac{\partial z_k}{\partial W_{ij}} = \sum_{d=1}^{D} x_d \frac{\partial}{\partial W_{ij}} W_{kd} \tag{59}$$

where

$$\frac{\partial}{\partial W_{ij}} W_{kd} = \begin{cases} 1 & \text{if } i=k, j=d \\ 0 & \text{else} \end{cases} \tag{60}$$

Therefore, since d is iterating through all elements, the sum will only be nonzero if $k=i$. That is, $\frac{\partial z_k}{\partial W_{ij}} = x_j$ if $k=i$ and 0 if else.

$$\frac{\partial z}{\partial W_{ij}} = \begin{bmatrix} 0 \\ \vdots \\ 0 \\ x_j \\ 0 \\ \vdots \\ 0 \end{bmatrix} \leftarrow i\text{th element} \tag{61}$$

Now computing

$$\frac{\partial L}{\partial W_{ij}} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial W_{ij}} = (a-y)\frac{\partial z}{\partial W_{ij}} = \sum_{k=1}^{K}(a_k - y_k)\frac{\partial z_k}{\partial W_{ij}} = (a_i - y_i)x_j \tag{62}$$

To get $\partial L/\partial W_{ij}$ we want a matrix whose entry (i,j) is $(a_i - y_i)x_j$. This is simply the outer product as shown below. For the bias term, $\partial z/\partial b$ is simply the identity matrix.

$$\frac{\partial L}{\partial W} = (a-y)x^T, \quad \frac{\partial L}{\partial b} = a - y \tag{63}$$

Therefore, summing the gradient over some minibatch $M \subset [N]$ gives

$$\nabla_W \ell_M = \sum_{i \in M} (h_\theta(x^{(i)}) - y^{(i)})(x^{(i)})^T, \quad \nabla_b \ell_M = \sum_{i \in M} (h_\theta(x^{(i)}) - y^{(i)}) \tag{64}$$

and our stochastic gradient descent algorithm is

$$\begin{aligned} \theta = \begin{pmatrix} W \\ b \end{pmatrix} &= \begin{pmatrix} W \\ b \end{pmatrix} - \eta \begin{pmatrix} \nabla_W \ell_M \\ \nabla_b \ell_M \end{pmatrix} \\ &= \begin{pmatrix} W \\ b \end{pmatrix} - \eta \begin{pmatrix} \sum_{i \in M} (h_\theta(x^{(i)}) - y^{(i)})(x^{(i)})^T \\ \sum_{i \in M} (h_\theta(x^{(i)}) - y^{(i)}) \end{pmatrix} \end{aligned}$$

Example 3.1 (SGD from Scratch)

```
>>> import numpy as np
>>> n, d, K = 500, 5, 3  # n samples, d features, K classes
>>> W_true = np.array([[1.0, 2.0, -1.5, 0.5, 3.0],
...                    [-2.0, 1.0, 2.5, -1.0, 0.5],
...                    [0.5, -1.5, 1.0, 2.0, -2.5]])
>>>
>>>
>>> b_true = np.array([0.5, -1.0, 1.5])
>>> X = np.random.randn(n, d)
>>> def softmax(z):
...   exp_z = np.exp(z - np.max(z, axis=1, keepdims=True))  # numerical stability
...   return exp_z / np.sum(exp_z, axis=1, keepdims=True)
...
>>>
>>> logits = X @ W_true.T + b_true
>>> probs = softmax(logits)
>>> Y_labels = np.array([np.random.choice(K, p=prob) for prob in probs])
>>> Y = np.zeros((n, K)) # Convert to one-hot encoding
>>> Y[np.arange(n), Y_labels] = 1
>>> W_hat, b_hat  = np.random.randn(K, d) * 0.1, np.random.randn(K) * 0.1
>>> lr = 0.05
>>> for epoch in range(10000):
...   logits = X @ W_hat.T + b_hat
...   predictions = softmax(logits)
...   error = predictions - Y  # shape: (n, K)
...   grad_W = error.T @ X / n  # shape: (K, d)
...   grad_b = np.mean(error, axis=0)  # shape: (K,)
...   W_hat -= lr * grad_W
...   b_hat -= lr * grad_b
...
>>> final_predictions = softmax(X @ W_hat.T + b_hat)
>>> final_accuracy = np.mean(np.argmax(final_predictions, axis=1)==Y_labels)
>>> print(W_hat)
[[ 1.33704007  1.96924581 -2.67632699 -0.09619256  3.33028542]
 [-2.12103603  0.49754612  2.08725926 -1.6633942  -0.24173753]
```

```
 [ 0.54933076 -2.50510753  0.39990491  1.88095809 -3.14859809]]
>>> print(b_hat)
[-0.01252576 -1.10961749  1.37124823]
>>> print("Accuracy:", final_accuracy)
Accuracy: 0.914
```

3.2 Significance Tests and Confidence Sets

3.3 Concentration Bounds

4 Regularized Softmax Regression

4.1 Ridge

In the high dimensional case, we would like to impose some regularization again to control variance.

Definition 4.1 (Loss)

The loss function of a ridge logistic regression is

$$L(\beta, x, y) = -y \log(\sigma(\beta^T x)) - (1-y)\log(1-\sigma(\beta^T x)) \tag{65}$$
$$= -y \log(\hat{y}) - (1-y)\log(1-\hat{y}) \tag{66}$$

The loss for a ridge softmax regression is

$$L(\theta, x, y) = -\sum_{k=1}^{K} y_k \log() \tag{67}$$

4.2 Lasso

5 Gaussian Discriminant Analysis

Another intuitive way to model binary data is to assume that the distribution of each class is a Gaussian. This allows us to work with the likelihood, and depending on which likelihood is greater, we can classify it at such.

Gaussian discriminant analysis in general is not actually a linear model. It turns out that because the two Gaussians can have different covariance matrices, we end up getting a quadratic boundary. However, if we restrict the two to have the *same* covariance matrix, we end up with *linear discriminant analysis*.

Definition 5.1 (Linear Discriminant Analysis)

The **LDA model** is a generative binary classification model that assumes the data is generated as

$$y \sim \text{Bernoulli}(\pi) \tag{68}$$
$$x \mid y = 0 \sim \mathcal{N}(\mu_0, \Sigma) \tag{69}$$
$$x \mid y = 1 \sim \mathcal{N}(\mu_1, \Sigma) \tag{70}$$

where the parameters are $\theta = \{\pi, \mu_0, \mu_1, \Sigma\}$. Sometimes, the shared covariance matrix Σ is assumed to be isotropic. This results in

$$f(x) = \begin{cases} 0 & \text{if } p(x \mid y = 0) \geq p(x \mid y = 1) \\ 1 & \text{if else} \end{cases} \tag{71}$$

Note that since we are creating a model of how the data is—not only distributed—but *generated*, we can also generate new data.

Theorem 5.1 (Decision Boundary is Linear)

Proof.

5.1 Comparison to Logistic Regression

Note that we can equivalently view LDA as a plug-in classifier with the Bayes rule. Let our posterior distribution be

$$p(y = 1 \mid x) = \frac{p(x \mid y = 1)\, \pi}{p(x \mid y = 0)\, (1 - \pi) + p(x \mid y = 1)\, \pi} \tag{72}$$

and we can threshold this to be $\frac{1}{2}$.

5.2 Maximum Likelihood Estimate

Lemma 5.1 (Likelihood)

The likelihood of a sample is

$$p(y) = \pi^y (1 - \pi)^{1-y} \tag{73}$$
$$p(x \mid y = 0) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2} (x - \mu_0)^T \Sigma^{-1} (x - \mu_0) \right) \tag{74}$$
$$p(x \mid y = 1) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2} (x - \mu_1)^T \Sigma^{-1} (x - \mu_1) \right) \tag{75}$$

Proof.

Theorem 5.2 (MLE)

The maximum likelihood estimate of LDA is

$$\pi = \frac{1}{N}\sum_{n=1}^{N} 1\{y^{(n)} = 1\} \tag{76}$$

$$\boldsymbol{\mu}_0 = \frac{\sum_{n=1}^{n} 1_{\{y^{(n)}=0\}}\mathbf{x}^{(n)}}{\sum_{n=1}^{N} 1_{\{y^{(n)}=0\}}} \tag{77}$$

$$\boldsymbol{\mu}_1 = \frac{\sum_{n=1}^{n} 1_{\{y^{(n)}=1\}}\mathbf{x}^{(n)}}{\sum_{n=1}^{N} 1_{\{y^{(n)}=1\}}} \tag{78}$$

$$\boldsymbol{\Sigma} = \frac{1}{N}\sum_{n=1}^{N}(\mathbf{x}^{(n)} - \mu_{y^{(n)}})(\mathbf{x}^{(n)} - \mu_{Y^{(i)}})^T \tag{79}$$

Proof.

We optimize $\pi \in (0,1)\mathbb{R}, \mu_0 \in \mathbb{R}^d, \mu_1 \in \mathbb{R}^d, \Sigma \in \text{Mat}(d \times d, \mathbb{R}) \simeq \mathbb{R}^{d\times d}$ so that we get the best-fit model. Assuming that each sample has been picked independently, this is equal to maximizing

$$L(\pi, \mu_0, \mu_1, \Sigma) = \prod_{i=1}^{n} \mathbb{P}\big(x^{(i)}, y^{(i)}\,;\, \pi, \mu_0, \mu_1, \Sigma\big) \tag{80}$$

which is really just the probability that we get precisely all these training samples $(x^{(i)}, y^{(i)})$ given the 4 parameters. This can be done by optimizing its log-likelihood, which is given by

$$l(\pi, \mu_0, \mu_1, \Sigma) = \log \prod_{i=1}^{n} \mathbb{P}(x^{(i)}, y^{(i)}; \pi, \mu_0, \mu_1, \Sigma) \tag{81}$$

$$= \log \prod_{i=1}^{n} \mathbb{P}(x^{(i)} \,|\, y^{(i)}; \mu_0, \mu_1, \Sigma)\, \mathbb{P}(y^{(i)}; \pi) \tag{82}$$

$$= \sum_{i=1}^{n} \log \Big(\mathbb{P}(x^{(i)} \,|\, y^{(i)}; \mu_0, \mu_1, \Sigma)\, \mathbb{P}(y^{(i)}; \pi) \Big) \tag{83}$$

A visual of the algorithm is below, with contours of the two Gaussian distributions, along with the straight line giving the decision boundary at which $\mathbb{P}(y = 1 \,|\, x) = 0.5$.

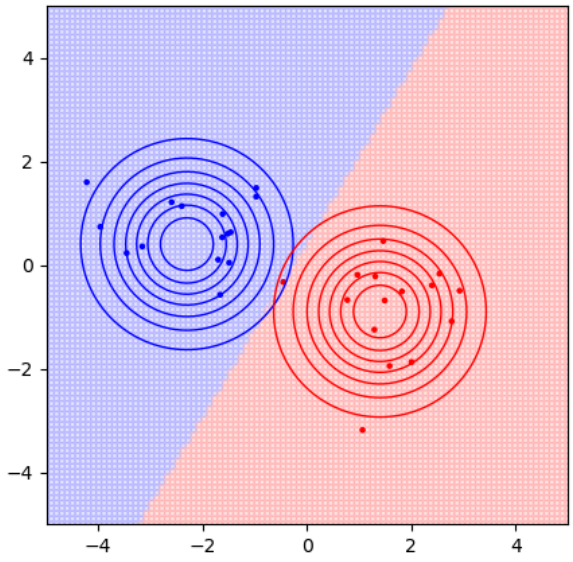


Figure 4: GDA of Data Generated from 2 Gaussisans centered at $(-2.3, 0.4)$ and $(1.4, -0.9)$ with unit covariance. The decision boundary is slightly off since MLE approximates the true means.

5.3 Quadratic Discriminant Analysis

Now let's do the general form.

Definition 5.2 (Gaussian Discriminant Analysis)

The **GDA model** is a generative binary classification model that assumes the data is generated as

$$y \sim \text{Bernoulli}(\pi) \tag{84}$$
$$x \mid y = 0 \sim \mathcal{N}(\mu_0, \Sigma_0) \tag{85}$$
$$x \mid y = 1 \sim \mathcal{N}(\mu_1, \Sigma_1) \tag{86}$$

where the parameters are $\theta = \{\pi, \mu_0, \mu_1, \Sigma_0, \Sigma_1\}$. This results in

$$f(x) = \begin{cases} 0 & \text{if } p(x \mid y = 0) \geq p(x \mid y = 1) \\ 1 & \text{if else} \end{cases} \tag{87}$$

Theorem 5.3 (Maximum Likelihood Estimate)

We can simplify the computation to the thresholding.

Theorem 5.4 (Bayes Rule for Two Gaussian Classes)

If $X \mid Y = 0 \sim N(\mu_0, \Sigma_0)$ and $X \mid Y = 1 \sim N(\mu_1, \Sigma_1)$, then the Bayes rule is

$$f^*(x) = \begin{cases} 1 & \text{if } r_1^2 < r_0^2 + 2\log\left(\frac{\pi_1}{1-\pi_1}\right) + \log\left(\frac{|\Sigma_0|}{|\Sigma_1|}\right) \\ 0 & \text{otherwise} \end{cases} \tag{88}$$

where $r_i^2 = (x-\mu_i)^T\Sigma_i^{-1}(x-\mu_i)$ for $i = 1, 2$ is the Mahalanobis distance.

Proof.

By definition, the Bayes rule is $h^*(x) = I(\pi_1 p_1(x) > (1-\pi_1)p_0(x))$. Plug-in the specific forms of p_0 and p_1 and take the logarithms we get $h^*(x) = 1$ if and only if

$$(x-\mu_1)^T\Sigma_1^{-1}(x-\mu_1) - 2\log\pi_1 + \log(|\Sigma_1|) \tag{89}$$

$$< (x-\mu_0)^T\Sigma_0^{-1}(x-\mu_0) - 2\log(1-\pi_1) + \log(|\Sigma_0|). \tag{90}$$

The theorem immediately follows from some simple algebra.

Theorem 5.5 (Decision Boundary is Quadratic)

Proof.

5.4 Multiclass GDA

Theorem 5.6 (Optimality of the Bayes Classifier)

Let $R(f) = \mathbb{P}(f(X) \neq Y)$ be the classification error of a classification rule $f(x)$. The Bayes rule $f^*(X)$ minimizing $R(f)$ can be written as

$$f^*(x) = \underset{k}{\operatorname{argmax}}\, \mathbb{P}(Y = k \mid X = x) \tag{91}$$

Proof.

We have

$$R(f) = 1 - \mathbb{P}(f(X) = Y) \tag{92}$$

$$= 1 - \sum_{k=0}^{K-1} \mathbb{P}(f(X) = k, Y = k) \tag{93}$$

$$= 1 - \sum_{k=0}^{K-1} \mathbb{E}\left[I(f(X) = k)\mathbb{P}(Y = k \mid X)\right] \tag{94}$$

It's clear that $f^*(X) = \operatorname{argmax}_k \mathbb{P}(Y = k \mid X)$ achieves the minimized classification error $1 - \mathbb{E}[\max_k \mathbb{P}(Y = k \mid X)]$.

Let $\pi_k = \mathbb{P}(Y = k)$. The next theorem extends QDA and LDA to the multiclass setting.

Theorem 5.7 (Multiclass QDA)

Suppose that $Y \in \{0, \dots, K-1\}$ with $K \geq 2$. If $p_k(x) = p(x \mid Y = k)$ is Gaussian: $X \mid Y = k \sim N(\mu_k, \Sigma_k)$, the Bayes rule for the multiclass QDA can be written as

$$f^*(x) = \underset{k}{\operatorname{argmax}}\, \delta_k(x) \tag{95}$$

where

$$\delta_k(x) = -\frac{1}{2}\log|\Sigma_k| - \frac{1}{2}(x-\mu_k)^T\Sigma_k^{-1}(x-\mu_k) + \log\pi_k. \tag{96}$$

If all Gaussians have an equal variance Σ, then

$$\delta_k(x) = x^T\Sigma^{-1}\mu_k - \frac{1}{2}\mu_k^T\Sigma^{-1}\mu_k + \log\pi_k. \tag{97}$$

Let $n_k = \sum_i \mathbb{1}(y_i = k)$ for $k = 0, \dots, K-1$. The estimated sample quantities of π_k, μ_k, Σ_k, and Σ are:

$$\hat{\pi}_k = \frac{1}{n}\sum_{i=1}^{n}\mathbb{1}(y_i = k), \quad \hat{\mu}_k = \frac{1}{n_k}\sum_{i:Y_i=k} X_i, \tag{98}$$

$$\hat{\Sigma}_k = \frac{1}{n_k - 1}\sum_{i:Y_i=k}(X_i - \hat{\mu}_k)(X_i - \hat{\mu}_k)^T, \tag{99}$$

$$\hat{\Sigma} = \frac{\sum_{k=0}^{K-1}(n_k - 1)\hat{\Sigma}_k}{n - K}. \tag{100}$$

6 Linear Support Vector Machines

Remember that it was the assumption that the conditional distribution $Y \mid X = x$ being Bernoulli/multinomial, along with the power form of the surrogate likelihood, that led to the likelihood derivation of the surrogate loss of the logistic and softmax regression. But ultimately, these are just modeling assumptions, and we may look for other surrogate losses to construct our risk.

If we want to construct such a loss function, we first want it to approximate the 0-1 step function clearly. The next thing is that we want it to be convex, so we can use convex optimization (sum of convex functions are convex). Perhaps we can think of the *smallest* convex function that stays above the binary loss, in some sense the best approximation. This is precisely the hinge function.

Definition 6.1 (Hinge Loss)

The **hinge loss** is a convex surrogate loss function for the 0-1 loss function. It is defined as

$$L(y, \hat{y}) = \max(0, 1 - y \cdot \hat{y}) \tag{101}$$

Note that we have directly constructed a loss without any reference to the likelihood of the data points. Now the SVM model is really simple. We just have a linear model $g(x) = \beta^T x$, and then use it to make a plug-in classifier through a threshold function with threshold 0.

Definition 6.2 (Binary Support Vector Machine)

The **binary SVM** is a linear classifier that sets

$$f(x) = \text{sign}(F(x)) = \text{sign}(\beta^T x) = \begin{cases} 1 & \text{if } \beta^T x \geq 0 \\ 0 & \text{else} \end{cases} \tag{102}$$

With the model and loss set up, we can define our risk.

Theorem 6.1 (Risk)

The expected risk is

$$R(f) = \mathbb{E}_{x,y}\left[\max\{0, 1 - yF(x)\}\right] = \mathbb{E}_{x,y}\left[\max\{0, 1 - y(\beta^T x)\}\right] \tag{103}$$

and the empirical risk is

$$\hat{R}(f) = \frac{1}{n}\sum_{i=1}^{n} \max\{0, 1 - y(\beta^T x)\} \tag{104}$$

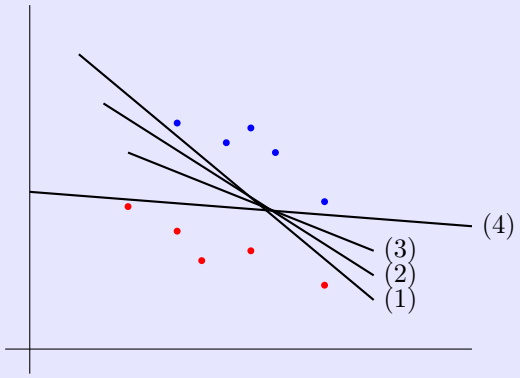
Note that if we classified something correctly, then $y(\beta^T x)$ would be positive, leading to the loss term being cut off to 0.[2] So the model will focus only on the points that are wrong or that are most difficult to tell apart, which are called the *support vectors*. The other points "far away" from the decision boundary have little to no effect on the optimal solution, and this is a big difference between SVMs and other classifiers.

Example 6.1 (SVMs vs Other Classifiers on Linearly Separable Dataset)

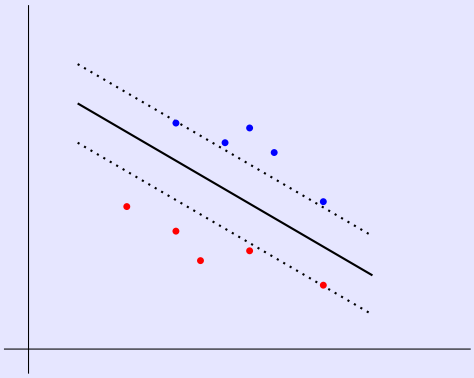
Assume that our dataset $\mathcal{D} = \{\mathbf{x}_i, y_i\}$ is linearly separable with $y_i \in \{-1, +1\}$. Based on previous algorithms like the perceptron, it will find some separating hyperplane. However, there's an infinite number of separating hyperplanes as shown in Figure 5a. What support vector machines want to do is to find the best one, with the "best" defined as the hyperplane that maximizes the distance between

[2] Unless $\beta^T x$ was a very small positive number, but speaking loosely here.

either the closest positive or negative samples, shown in Figure 5*b*.



(a) Planes such as (1) and (4) are "too close" to the positive and negative samples.



(b) SVMs try to find the separating hyperplane with the best minimum margin.

Figure 5: Motivating problem

We want to formalize the concepts of these margins that we wish to maximize. Furthermore, this problem is not well defined since we can just set w to be 0 or an arbitrarily high norm vector, which makes this problem ill-posed. Therefore, we will need some constraints as well.

6.1 Functional and Geometric Margins

To do this, we will define two terms.

Definition 6.3 (Geometric margin)

Given a point $\mathbf{x}_0$ and a hyperplane of equation $\mathbf{w} \cdot \mathbf{x} + b = 0$, the distance from $\mathbf{x}_0$ to the hyperplane, known as the **geometric margin**, can be computed with the formula

$$d = \frac{|\mathbf{x}_0 \cdot \mathbf{w} + b|}{\|\mathbf{w}\|} \tag{105}$$

Therefore, the geometric margin of the ith sample with respect to the hypothesis f is defined

$$\gamma_i = \frac{y_i \left(\mathbf{w} \cdot \mathbf{x}_i + b\right)}{\|\mathbf{w}\|} \tag{106}$$

We wish to optimize the parameters $\mathbf{w}, b$ in order to maximize the minimum of the geometric margins (the distance between the closest point and the hyperplane).

$$\underset{\mathbf{w},b}{\operatorname{argmax}} \min_i \gamma_i = \underset{\mathbf{w},b}{\operatorname{argmax}} \left\{ \frac{1}{\|\mathbf{w}\|} \min_i \left[y_i \left(\mathbf{w} \cdot \mathbf{x}_i + b\right) \right] \right\} \tag{107}$$

Direct solution of this optimization problem would be very complex, and so we convert this into an equivalent problem that is much easier to solve. Note that the solution to the above term is not unique. If there was a solution $(\mathbf{w}^*, b^*)$, then

$$\frac{y_i(\mathbf{w} \cdot \mathbf{x}_i + b)}{\|\mathbf{w}\|} = \frac{y_i(\lambda\mathbf{w} \cdot \mathbf{x}_i + \lambda b)}{\|\lambda\mathbf{w}\|} \tag{108}$$

That is, the geometric margin is not sensitive to scaling of the parameters of the hyperplane. Therefore, we can scale the numerator and the denominator by whatever we want and use this freedom to set

$$y_i(\mathbf{w} \cdot \mathbf{x}_i + b) = 1$$

for the point that is closest to the surface. In that case, all data points will satisfy the constraints

$$y_n(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1$$

In the case of data points for which the equality holds, the constraints are said to be *active*, whereas for the remainder they are *inactive*. Therefore, it will always be the case that $\min_i \left[y_i \left(\mathbf{w} \cdot \mathbf{x}_i + b\right)\right] = 1$, and the constraint problem reduces to

$$\underset{\mathbf{w},b}{\text{argmax}} \frac{1}{\|\mathbf{w}\|} = \underset{\mathbf{w},b}{\text{argmin}} \frac{1}{2}\|\mathbf{w}\|^2 \text{ subject to constraints } y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 \tag{109}$$

This final step is the most significant step in this derivation and may be hard to wrap around the first time. So we dedicate the next subsubsection for this.

We could just work straight with this geometric margin, but for now, let's try to extend what we did with the perceptron into SVMs. We will find out that extending the concept of functional margins into SVMs leads to ill-defined problems. In the perceptron, we wanted to construct a function $f(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b$ such that

$$y_i\, f(\mathbf{x}_i) \geq 0 \text{ for all } i = 1, 2, \ldots, N$$

Definition 6.4 (Functional Margin)

The value of $y_i\, f(\mathbf{x}_i)$ gives us our confidence on our classification, and in a way it represents a kind of distance away from the separating hyperplane (if this value was 0, then we would be 50 50 split on whether to label it positive or negative). Therefore, we shall define

$$\hat{\gamma}_i = y_i f(\mathbf{x}_i)$$

as the **functional margin** of $(\mathbf{w}, b)$ with respect to the training sample $(\mathbf{x}_i, y_i)$. Therefore, the smallest of the function margins can be written

$$\hat{\gamma} = \min_i \gamma_i$$

called the **function margin**.

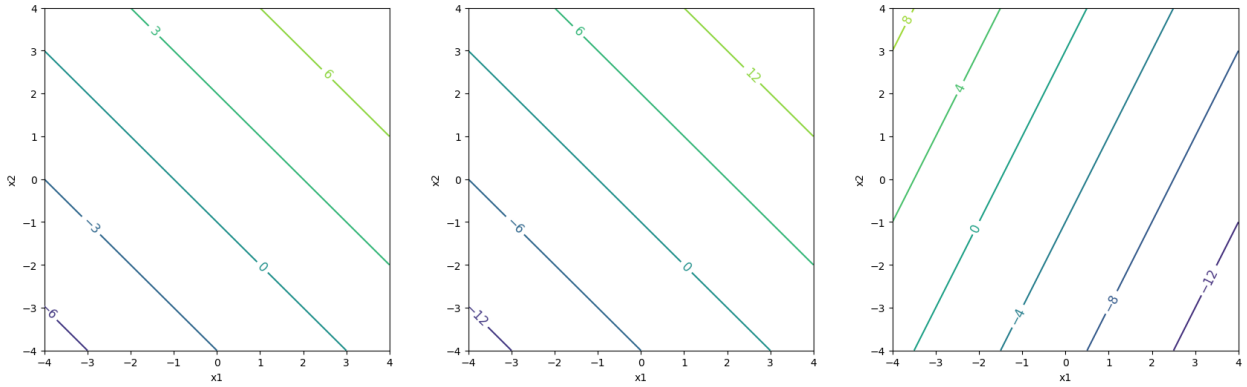
Note that the geometric margin and functional margin are related by a constant scaling factor. Given a sample $(\mathbf{x}_i, y_i)$, we have

$$\text{GeometricMargin} = \frac{y_i\left(\mathbf{w} \cdot \mathbf{x}_i + b\right)}{\|\mathbf{w}\|_2} = \frac{\text{FunctionalMargin}}{\|\mathbf{w}\|_2}$$

As we can see, the perceptron works with the functional margin, and since it does not care about how large the margin is (just whether it's positive or negative), we are left with an underdetermined system in which there exists infinite $(\mathbf{w}, b)$'s. Now what we want to do is impose a certain minimum margin $\gamma > 0$ and solve for $(\mathbf{w}, b)$ again, and keep increasing this γ until there is some unique solution. We can view this problem in two ways:

1. Take a specific minimum margin γ and find a $(\mathbf{w}, b)$, which may not exist, be unique, or exist infinitely that satisfies
$$y_i f(\mathbf{x}) = y_i(\mathbf{w} \cdot \mathbf{x} + b) \geq \gamma \text{ for all } i = 1, \ldots, N$$
2. Take a specific $(\mathbf{w}, b)$ and calculate the maximum γ that satisfies the constraint equations above.

They're both equivalent problems, but both ill-posed if we look at (2). Since the samples are linearly separable by assumption, we can say that there exists some $\epsilon > 0$ such that $y_i f(\mathbf{x}_i) \geq \epsilon$ for all i. Therefore, if we just scale $(\mathbf{w}, b) \mapsto (\lambda\mathbf{w}, \lambda b)$ for some large λ, this leads to the solution for γ being unbounded. We can see in Figure 6 that we can increased confidence at no cost. Looking at (1), we can also see that if $(\mathbf{w}, b)$ does exist, then every other $(\lambda\mathbf{w}, \lambda b)$ for $\lambda > 1$ satisfies the property.



(a) $f(x) = x_1 + x_2 + 1$ (b) $f(x) = 2x_1 + 2x_2 + 2$ (c) $f(x) = -2x_1 + x_2 - 3$

Figure 6: From (a), you can see that simply multiplying everything by two automatically increases our confidence by 2, meaning that the functional margin can be scaled arbitrarily by scaing $(\mathbf{w}, b)$. There are still too many degrees of freedom in here and so extra constraints must be imposed.

6.2 Analytical Solution

To minimize the equations with the constraint equations, we can use the method of Lagrange multipliers, which leads to to Lagrangian

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2}\|\mathbf{w}\|^2 - \sum_i \alpha_i \big[y_i(\mathbf{w} \cdot \mathbf{x}_i + b) - 1\big] \tag{110}$$

We can take the gradients with respect to $\mathbf{w}$ and b and set them to 0, which gives the two conditions

$$\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$$
$$0 = \sum_i \alpha_i y_i \mathbf{x}_i$$

Now let's substitute our evaluated $\mathbf{w}$ back into $\mathcal{L}$, which gives the **dual representation** of the maximum margin problem in which we maximize

$$\begin{aligned} L &= \frac{1}{2}\left(\sum_i \alpha_i y_i \mathbf{x}_i\right)\left(\sum_j \alpha_j y_j \mathbf{x}_j\right) - \sum_i \alpha_i y_i x_i \cdot \left[\sum_j \alpha_j y_j x_j\right] - \sum_i \alpha_i y_i b + \sum_i \alpha_i \\ &= \sum_i \alpha_i - \frac{1}{2}\sum_{i,j} \alpha_i \alpha_j y_i y_j \, \mathbf{x}_i \cdot \mathbf{x}_j \end{aligned}$$

The summation with the b in it is 0 since we can pull the b out and the remaining sum is 0 from before. Now the optimization only depends on the dot product $\mathbf{x}_i \cdot \mathbf{x}_j$ of all pairs of sample vectors, which is very interesting. We will see more of this when we talk about kernel methods. Now, we need to solve the dual problem

$$\max_{\boldsymbol{\alpha}} \mathcal{L}(\boldsymbol{\alpha}) \tag{111}$$

which can be done using some generic quadratic programming solver or some other method to get the optimum $\boldsymbol{\alpha}^*$, which gives us

$$\mathbf{w}^* = \sum_i \alpha_i^* y_i \mathbf{x}_i \tag{112}$$

6.3 Significance Tests and Confidence Sets

6.4 Concentration Bounds

6.5 Nonseparable Case

7 Generalized Linear Models

Remember the linear model looked like this, where we use the conventional β notation to represent parameters.

$$Y = X^T\beta + \epsilon, \quad \epsilon \sim N(0, \sigma^2 I) \tag{113}$$

which implies that $Y \mid X \sim N(X^T\beta, \sigma^2 I)$. Basically, given x, I assume some distribution of Y, and the value of x will help me guess what the mean of this distribution is. Note that we in here assume that only the mean depends on X. I could potentially have something crazy, like

$$Y \mid X \sim N(X^T\beta, (X^T\gamma)(XX^T + I)) \tag{114}$$

where the covariance will depend on X, too, but in this case we only assume that that mean is dependent on X.

$$Y \mid X \sim N(\mu(X), \sigma^2 I) \tag{115}$$

where in the linear model, $\mu(X) = X^T\beta$. So, there are three assumptions we are making here:

1. $Y \mid X$ is Gaussian.
2. X only affects the mean of $Y \mid X$, written $\mathbb{E}[Y \mid X] = \mu(X)$.
3. X affects the mean in a linear way, such that $\mu(X) = X^T\beta$.

So the two things we are trying to relax are:

1. **Random Component**: the response variable $Y \mid X$ is continuous and normally distributed with mean $\mu = \mu(X) = \mathbb{E}[Y \mid X]$.
2. **Link**: I have a link that explains the relationship between the X and the μ, and this relationship is $\mu(X) = X^T\beta$.

So when talking about GLMs, we are not changing the fact that we have a linear function $X \mapsto X^T\beta$. However, we are going to assume that $Y \mid X$ now comes from a broader **family of exponential distributions**. Second, we are going to assume that there exists some **link function** g

$$g(\mu(X)) = X^T\beta \tag{116}$$

Admittedly, this is not the most intuitive way to think about it, since we would like to have $\mu(X) = f(X^T\beta)$, but here we just decide to call $f = g^{-1}$. Therefore, if I want to give you a GLM, I just need to give you two things: the conditional distribution $Y \mid X$, which can be any distribution in the exponential family, and the link function g.

We really only need this link function due to compatibility reasons. Say that $Y \mid X \sim \text{Bern}(p)$. Then, $\mu(X) = \mathbb{E}[Y \mid X]$ always lives in $[0, 1]$, but $X^T\beta$ always lives in $\mathbb{R}$. We want our model to be realistic, and we can clearly see the problem shown in Figure 7.

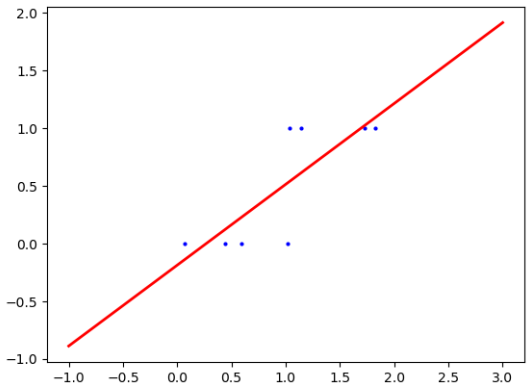


Figure 7: Fitting a linear model for Bernoulli random variables will predict a mean that is outside of $[0, 1]$ when getting new datapoints.

If $Y \mid X$ is some exponential distribution, then its support is always positive and so $\mu(X) > 0$. But if we stick to the old form of $\mu(X) = X^T\beta$, then $\text{Im}(\mu) = \mathbb{R}$, which is not realistic when we predict negative values. Let's take a couple examples:

Example 7.1 (Disease Epidemic)

In the early stages of a disease epidemic, the rate at which new cases occur can often increase exponentially through time. Clearly, $\mu(X) = \mathbb{E}[Y \mid X]$ should be positive and we should have some sort of exponential trend. Hence, if $\mu(x)$ is the expected number of cases on data x, a model of the form

$$\mu(x) = \gamma \exp(\delta x) \tag{117}$$

seems appropriate, where γ and δ are simply scaling factors. Clearly, $\mu(X)$ is not of the form $f(X^T\beta)$. So what I do is to transform μ in such a way that I can get something that is linear.

$$\log(\mu(X)) = \log(\gamma) + \delta X \tag{118}$$

which is now linear in X, of form $\beta_0 + \beta_1 X$. This will have some effects, but this is what needs to be done to have a genearlized linear model. Note that what I did to μ was take the log of it, and so the link function is $g = \log$, called the **log-link**. Now that we have chosen the g, we still need to choose what the conditional distribution $Y \mid X$ would be. This is determined by speaking with industry professionals, experience, and convenience. In this case, Y is a count, and since this must be a discrete distribution. Since it is not bounded above, we think Poisson.

Example 7.2 (Prey Capture Rate)

The rate of capture of preys, Y, by a hunting animal, tends to increase with increasing density of prey X, but eventually level off when the predator is catching as much as it can cope with. We want to find a perhaps concave function that levels off, and suitable model might be

$$\mu(X) = \frac{\alpha X}{h + X} \tag{119}$$

where α represents the maximum capture rate, and h represents the prey density at which the capture rate is half the maximum rate. Again, we must find some transformation g that turns this into a

linear function of X, and what we can do it use the **reciprocal-link**.

$$\frac{1}{\mu(X)} = \frac{h+X}{\alpha X} = \frac{h}{\alpha}\frac{1}{X} + \frac{1}{\alpha} \tag{120}$$

The standard deviation of capture rate might be approximately proportional to the mean rate, suggesting the use of a Gamma distribution for the response.

Example 7.3 (Kyphosis Data)

The Kyphosis data consist of measurements on 81 children following corrective spinal surgery. The binary response variable, Kyphosis, indicates the presence or absence of a postoperative deforming. The three covariates are: age of the child in months, number of the vertebrae involved in the operation, and the start of the range of the vertebrae involved. The response variable is binary so there is no choice: $Y \mid X$ is Bernoulli with expected value $\mu(X) \in (0,1)$. We cannot write $\mu(X) = X^T\beta$ because the right hand side ranges through $\mathbb{R}$, and so we find an invertible function that squishes $\mathbb{R}$ to $(0,1)$, and so we can choose basically any CDF.

For clarification, when writing a distribution like Bernoulli(p), or Binomial(n,p), Poisson(λ), or $N(\mu,\sigma^2)$, the hyperparameters that we usually work with we will denote as $\boldsymbol{\theta}$, and the space that this $\boldsymbol{\theta}$ lives in will denote Θ. For example, for the Bernoulli, $\Theta = [0,1]$, and for Poisson, $\Theta = [0,+\infty)$.

Ultimately, a GLM consists of three steps:

1. The observed input X enters the model through a linear function $\beta^T X$.
2. The conditional mean of response, is represented as a function of the linear combination

$$\mathbb{E}[Y \mid X] = \mu = f(\beta^T X) \tag{121}$$

3. The observed response is drawn from an exponential family distribution with conditional mean μ.

7.1 Exponential Family

We can write the pdf of a distribution as a function of the input x and the hyperparameters θ, so we can write $P_\theta(x) = p(\theta, x)$. For now, let's think that both $x, \theta \in \mathbb{R}$. Think of all the functions that depend on θ and x. There are many of them, but we want θ and x to interact in a certain way. The way that I want them to interact with each other is that they are multiplied within an exponential term. Now clearly, this is not a very rich family, so we are just slapping some terms that depend only on θ and only on x.

$$p_\theta(x) = \exp(\theta x)h(x)c(\theta) \tag{122}$$

But now if $\theta \in \mathbb{R}^k$ and $x \in \mathbb{R}^q$, then we cannot simply take the product nor the inner product, but what we can do is map both of them into a space that has the same dimensions, so I can take the inner product. That is, let us map $\boldsymbol{\theta} \mapsto \boldsymbol{\eta}(\boldsymbol{\theta}) \in \mathbb{R}^k$ and $\mathbf{x} \mapsto \mathbf{T}(\mathbf{x}) \in \mathbb{R}^k$, and so our exponential distribution form would be generalized into something like

$$p_{\boldsymbol{\theta}}(\mathbf{x}) = \exp\left[\boldsymbol{\eta}(\boldsymbol{\theta}) \cdot \mathbf{T}(\mathbf{x})\right] h(\mathbf{x}) c(\boldsymbol{\theta}) \tag{123}$$

We can think of $c(\boldsymbol{\theta})$ as the normalizing term that allows us to integrate the pdf to 1.

$$\int_{\mathcal{X}} p_{\boldsymbol{\theta}}(\mathbf{x}) = c(\boldsymbol{\theta}) \int \exp\left[\boldsymbol{\eta}(\boldsymbol{\theta}) \cdot \mathbf{T}(\mathbf{x})\right] h(\mathbf{x})\, d\mathbf{x} \tag{124}$$

We can just push the $c(\boldsymbol{\theta})$ term into the exponential by letting $c(\boldsymbol{\theta}) = e^{-\log(c(\boldsymbol{\theta}))^{-1}}$ to get our definition.

Definition 7.1 (Exponential Family)

A **k-parameter exponential family** is a family of distributions with pdf/pmf of the form

$$p_{\boldsymbol{\theta}}(\mathbf{x}) = \exp\left[\boldsymbol{\eta}(\boldsymbol{\theta}) \cdot \mathbf{T}(\mathbf{x}) - B(\boldsymbol{\theta})\right] h(\mathbf{x}) \tag{125}$$

The h term, as we will see, will not matter in our maximum likelihood estimation, so we keep it outside the exponential.

1. $\boldsymbol{\eta}$ is called the **canonical parameter**. Given a distribution parameterized by the regular hyperparameters $\boldsymbol{\theta}$, we would like to parameterize it in a different way $\boldsymbol{\eta}$ under the function $\boldsymbol{\eta} : \Theta \to \mathbb{R}$
2. $\mathbf{T}(\mathbf{x})$ is called the **sufficient statistic**.
3. $h(\mathbf{x})$ is a nonnegative scalar function.
4. $B(\boldsymbol{\theta})$ is the normalizing factor.

Let's look at some examples.

Example 7.4 (Gaussian)

If we put the coefficient into the exponential and expand the square term, we get

$$p_\theta(x) = \exp\left(\frac{\mu}{\sigma^2} \cdot x - \frac{1}{2\sigma^2} \cdot x^2 - \frac{\mu^2}{2\sigma^2} - \log(\sigma\sqrt{2\pi})\right) \tag{126}$$

where

$$\boldsymbol{\eta}(\boldsymbol{\theta}) = \begin{pmatrix} \mu/\sigma^2 \\ -1/2\sigma^2 \end{pmatrix},\ T(x) = \begin{pmatrix} x \\ x^2 \end{pmatrix},\ B(\theta) = \frac{\mu^2}{2\sigma^2} + \log(\sigma\sqrt{2\pi}),\ h(x) = 1 \tag{127}$$

This is not a unique representation since we can take the $\log(\sqrt{2\pi})$ out of the exponential, but why bother to do this when we can just stuff everything into B and keep h simple.

Example 7.5 (Gaussian with Known Variance)

If we have known variance, we can write the Gaussian pdf as

$$p_\theta(x) = \exp\left[\frac{\mu}{\sigma} \cdot \frac{x}{\sigma} - \frac{\mu^2}{2\sigma^2}\right] \cdot \frac{1}{\sigma\sqrt{2\pi}} e^{x^2/2\sigma^2} \tag{128}$$

where

$$\eta(\theta) = \frac{\mu}{\sigma},\ T(x) = \frac{x}{\sigma},\ B(\theta) = \frac{\mu^2}{2\sigma^2},\ h(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{x^2/2\sigma^2} \tag{129}$$

Example 7.6 (Bernoulli)

The pmf of a Bernoulli with θ is

$$\begin{aligned} p_\theta(x) &= \theta^x (1-\theta)^{(1-x)} \\ &= \exp\left[x \log(\theta) + (1-x)\log(1-\theta)\right] \\ &= \exp\left(x \log\left[\frac{\theta}{1-\theta}\right] - \log\left[\frac{1}{1-\theta}\right]\right) \end{aligned}$$

where

$$\eta(\theta) = \log\left[\frac{\theta}{1-\theta}\right],\ T(x) = x,\ B(\theta) = \log\left[\frac{1}{1-\theta}\right],\ h(x) = 1 \tag{130}$$

Example 7.7 (Binomial with Known Number of Trials)

We can transform a binomial with known N as

$$\begin{aligned} p_\theta(x) &= \binom{N}{x} \theta^x (1-\theta)^{1-x} \\ &= \exp\left[x \ln\left(\frac{\theta}{1-\theta}\right) + \ln(1-\theta) \right] \cdot \binom{N}{x} \end{aligned}$$

where

$$\eta(\theta) = \ln\left(\frac{\theta}{1-\theta}\right),\ T(x) = x,\ B(\theta) = \ln(1-\theta),\ h(x) = \binom{N}{x} \tag{131}$$

Example 7.8 (Poisson)

The pmf of Poisson with θ can be expanded

$$\begin{aligned} p_\theta &= \frac{\theta^{-x}}{x!} e^{-\theta} \\ &= \exp\left[-\theta + x\log(\theta) - \log(x!) \right] \\ &= \exp\left[x\log(\theta) - \theta \right] \frac{1}{x!} \end{aligned}$$

where

$$\eta(\theta) = \log(\theta),\ T(x) = x,\ B(\theta) = \theta,\ h(x) = \frac{1}{x!} \tag{132}$$

However, the uniform is not in here. In fact, any distribution that has a support that does not depend on the parameter is not an exponential distribution.

Let us now focus on one parameter families where $\theta \in \Theta \subset \mathbb{R}$, which do not include the Gaussian (with unknown mean and variance, Gamma, multinomial, etc.), which has a pdf written in the form

$$p_\theta(x) = \exp\left[\eta(\theta)\, T(x) - B(\theta) \right] h(x) \tag{133}$$

7.1.1 Canonical Exponential Family

Now a common strategy in statistical analysis is to reparamaterize a probability distribution. Suppose a family of probability distributions $\{P_\theta\}$ is parameterized by $\theta \in \Theta \subset \mathbb{R}$. If we have an invertible function $\eta : \Theta \to \mathcal{T} \subset \mathbb{R}$, then we can paramaterize the same family with η rather than θ, with no loss of information. Typically, it is the case that η is invertible for exponential families, so we can just reparameterize the whole pdf and write

$$p_\eta(x) = \exp\left[\eta\, T(x) - \phi(\eta) \right] h(x) \tag{134}$$

where $\phi = B \circ \eta^{-1}$.

Definition 7.2 (Canonical One-Parameter Exponential Family)

A family of distributions is said to be in **canonical one-parameter exponential family** if its density is of form

$$p_\eta(x) = \exp\left[\eta\, T(x) - \phi(\eta) \right] h(x) \tag{135}$$

which is a subfamily of the exponential family. The function ψ is called the **cumulant generating function**.

Before we move on, let us just provide a few examples.

Example 7.9 (Poisson)

The Poisson can be represented as

$$p_\theta(x) = \exp\left[x \log \theta - \theta\right] \frac{1}{x!} \tag{136}$$

Now let $\eta = \log \theta \implies \theta = e^\eta$. So, we can reparamaterize the density as

$$p_\eta(x) = \exp\left[x\eta - e^\eta\right] \frac{1}{x!} \tag{137}$$

where $P_\eta = \text{Poisson}(e^\eta)$ for $\eta \in \mathcal{T} = \mathbb{R}$, compared to $P_\theta = \text{Poisson}(\theta)$ for $\theta \in \Theta = \mathbb{R}^+$.

Example 7.10 (Gaussian)

Recall that the Gaussian with known parameter σ^2 and unknown $\theta = \mu$ is in the exponential family, since we can expand it as

$$p_\theta(x) = \exp\left[\frac{\mu}{\sigma^2} \cdot x - \frac{\mu^2}{2\sigma^2}\right] \cdot \frac{1}{\sigma\sqrt{2\pi}} e^{x^2/2\sigma^2} \tag{138}$$

We can perform the change of parameter $\eta = \mu^2/2\sigma^2 \implies \mu = \sigma^2\eta$, and substituting this in will give the canonical representation

$$p_\eta(x) = \exp\left[\eta x - \frac{\sigma^2\eta^2}{2}\right] \cdot \frac{1}{\sigma\sqrt{2\pi}} e^{x^2/2\sigma^2} \tag{139}$$

where now $P_\eta = N(\sigma^2\eta, \sigma^2)$ for $\eta \in \mathcal{T} = \mathbb{R}$, compared to $P_\theta = N(\theta, \sigma^2)$ for $\theta \in \Theta = \mathbb{R}$, which is basically the same space.

Example 7.11 (Bernoulli)

The Bernoulli has an exponential form of

$$p_\theta(x) = \exp\left[x \log\left(\frac{\theta}{1-\theta}\right) + \log(1-\theta)\right] \tag{140}$$

Now setting $\eta = \log\left(\frac{\theta}{1-\theta}\right) \implies \theta = \frac{1}{1+e^{-\eta}}$, and so $B(\theta) = -\log(1-\theta) = -\log\left(\frac{e^{-\eta}}{1+e^{-\eta}}\right) = \log(1+e^\eta) = \psi(\eta)$, and so the canonical paramaterization is

$$p_\eta(x) = \exp\left[x\eta - \log(1+e^\eta)\right] \tag{141}$$

We present two useful properties of the exponential family.

Theorem 7.1 (Moments)

Let random variable X be in the canonical exponential family P_η

$$p_\eta(x) = e^{\eta T(x) - \psi(\eta)} h(x) \tag{142}$$

Then, the expectation and variance are encoded in the cumulant generating function in the following way

$$\mathbb{E}[T(X)] = \psi'(\eta) \qquad \text{Var}[T(X)] = \psi''(\eta) \tag{143}$$

Proof.

Example 7.12 (Poisson, Normal and Bernoulli as Exponential Families)

We show that this is consistent with the Poisson, normal, and Bernoulli distributions.

1. In the Poisson, $\psi(\eta) = e^\eta$, and so $\psi'(\eta) = e^\eta = \theta = \mathbb{E}[X]$. Taking the second derivative gives $\psi''(\eta) = e^\eta = \theta = \text{Var}[X]$, too.
2. In the Normal with known variance σ^2, we have $\psi(\eta) = \frac{1}{2}\sigma^2\eta^2$. So

$$\begin{aligned} \mathbb{E}[X] &= \psi'(\eta) = \sigma^2\eta = \mu \\ \text{Var}[X] &= \psi''(\eta) = \sigma^2 \end{aligned}$$

3. In the Bernoulli, we have $\psi(\eta) = \log(1 + e^{-\eta})$. Therefore,

$$\begin{aligned} \mathbb{E}[X] &= \psi'(\eta) = \frac{x^\eta}{1+x^\eta} = \frac{1}{1+e^{-\eta}} = \theta \\ \text{Var}[X] &= \psi''(\eta) = -\left(\frac{1}{1+e^{-\eta}}\right)^2 e^{-\eta} \cdot -1 = \theta^2 \cdot \frac{1-\theta}{\theta} = \theta(1-\theta) \end{aligned}$$

Theorem 7.2 (Convexity)

Consider a canonical exponential family with density

$$p_\eta(x) = e^{\eta T(x) - \psi(\eta)} h(x) \tag{144}$$

and natural parameter space $\mathcal{T}$. Then, the set $\mathcal{T}$ is convex, and the cumulant generating function ψ is convex on $\mathcal{T}$.

Proof.

This can be proven using Holder's inequality. However, from the theorem above, note that $\text{Var}[T(X)] = \psi''(\eta)$ must be positive since we are talking about variance. This implies that the second derivative of ψ is positive, and therefore must be convex.

We will look at a subfamily of the exponential family. Now remember that we introduce the functions $\boldsymbol{\eta}$ and $\mathbf{T}$ so that we can capture a much broader range of distributions, but if we have one parameter $k = 1$, then we can just set $\boldsymbol{\eta}(\boldsymbol{\theta})$ to be the new parameter θ. The **canonical exponential family** for $k = 1, y \in \mathbb{R}$, is defined to have the pdf

$$f_\theta(y) = \exp\left(\frac{y\theta - b(\theta)}{\phi} + c(y, \phi)\right) \tag{145}$$

where

$$h(y) = \exp\big(c(y, \phi)\big) \tag{146}$$

If ϕ is known, this is a one-parameter exponential family with θ being the **canonical parameter**, and if ϕ is unknown, the $h(y)$ term will not depend on θ, which we may not be able to split up into the exponential pdf form. In this case ϕ is called the **dispersion parameter**. For now, we will always assume that ϕ is known.

We can prove this for all other classes, too. We can think of the $c(y, \phi)$ as just a term that we stuff every other term into. What really differentiates the different distributions of the canonical exponential family is the $b(\theta)$. The form of b will determine whether this distribution is a Gaussian, or a Bernoulli, or etc. This b will capture information about the mean, the variance, the likelihood, about everything.

7.2 Cumulant Generating Function

Definition 7.3 (Score)

The **score** is the gradient of the log-likelihood function with respect to the parameter vector. That is, given that $L(\boldsymbol{\theta})$ is the likelihood, then

$$s(\boldsymbol{\theta}) \coloneqq \frac{\partial \log L(\boldsymbol{\theta}; \mathbf{x})}{\partial \boldsymbol{\theta}} \tag{147}$$

which gives a row covector.

Now, remember that the score also depends on the observations $\mathbf{x}$. If we rewrite the likelihood as a probability density function $L(\boldsymbol{\theta}; \mathbf{x}) = f(\mathbf{x}; \boldsymbol{\theta})$, then we can say that the expected value of the score is equal to 0, since

$$\begin{aligned}
\mathbb{E}[s(\boldsymbol{\theta})] &= \int_{\mathcal{X}} f(\mathbf{x}; \boldsymbol{\theta}) \frac{\partial}{\partial \boldsymbol{\theta}} \log L(\boldsymbol{\theta}; \mathbf{x}) \, d\mathbf{x} \\
&= \int_{\mathcal{X}} f(\mathbf{x}; \boldsymbol{\theta}) \frac{1}{f(\mathbf{x}; \boldsymbol{\theta})} \frac{\partial f(\mathbf{x}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \, dx \\
&= \frac{\partial}{\partial \boldsymbol{\theta}} \int_{\mathcal{X}} f(\mathbf{x}; \boldsymbol{\theta}) \, d\mathbf{x} \\
&= \frac{\partial}{\partial \boldsymbol{\theta}} 1 = 0
\end{aligned}$$

where we take a leap of faith in switching the derivative and integral in the penultimate line. Furthermore, we can get the second identity

$$\mathbb{E}\left[\frac{\partial^2 \ell}{\partial \theta^2}\right] + \mathbb{E}\left[\frac{\partial \ell}{\partial \theta}\right]^2 = 0 \tag{148}$$

We can apply these two identities as follows. Since

$$\ell(\theta) = \frac{Y\theta - b(\theta)}{\phi} + c(Y; \phi) \tag{149}$$

therefore

$$\frac{\partial \ell}{\partial \theta} = \frac{Y - b'(\theta)}{\phi} \tag{150}$$

which yields

$$0 = \mathbb{E}\left[\frac{\partial \ell}{\partial \theta}\right] = \frac{\mathbb{E}[Y] - b'(\theta)}{\phi} \implies \mathbb{E}[Y] = \mu = b'(\theta) \tag{151}$$

On the other hand, we have

$$\frac{\partial^2 \ell}{\partial \theta^2} + \left(\frac{\partial \ell}{\partial \theta}\right)^2 = -\frac{b''(\theta)}{\phi} + \left(\frac{Y - b'(\theta)}{\phi}\right)^2 \tag{152}$$

and from the previous result, we get

$$\frac{Y - b'(\theta)}{\phi} = \frac{Y - \mathbb{E}[Y]}{\phi} \tag{153}$$

together with the second identity, yields

$$0 = -\frac{b''(\theta)}{\phi} + \frac{\mathrm{Var}(Y)}{\phi^2} \implies \mathrm{Var}(Y) = \phi\, ''(\theta) \tag{154}$$

Since variance is always positive, this implies that $b'' > 0$ and therefore b must be convex.

7.3 Link Functions

Now let's go back to GLMs. In linear models, we said that the conditional expectation of Y given $X = \mathbf{x}$ must be a linear function in x

$$\mathbb{E}[Y \mid X = \mathbf{x}] = \mu(\mathbf{x}) = \mathbf{x}^T\beta \tag{155}$$

But if the conditional distribution takes values in some subset of $\mathbb{R}$, such as $(0, 1)$, then it may not make sense to write this as a linear function, since $X^T\beta$ has an image spanning $\mathbb{R}$. So what we need is a link function that relates, i.e. transforms the restricted subset of μ, onto the real line, so that now you can express it of the form $X^T\beta$.

$$g\big(\mu(X)\big) = X^T\beta \tag{156}$$

Again, it is a bit more intuitive to talk about g^{-1}, which takes our $X^T\beta$ and transforms it to the values that I want, so we will talk about both of them simultaneously. If g is our link function, we want it to satisfy 3 requirements:

1. g is continuously differentiable
2. g is strictly increasing
3. $\text{Im}(g) = \mathbb{R}$, i.e. it spans the entire real line

This implies that g^{-1} exists, which is also continuously differentable and is strictly increasing.

Example 7.13 (Choosing a Link Function)

If I have a conditional distribution...

1. that is Poisson, then we want our μ to be positive, and so we need a link function $g : \mathbb{R}^+ \to \mathbb{R}$. One choice would be the logarithm

$$g(\mu(X)) = \log\big(\mu(X)\big) = X^T\beta \tag{157}$$

2. that is Bernoulli, then we want our μ to be in $(0, 1)$ and we need a link function $g : (0, 1) \to \mathbb{R}$. There are 2 natural choices, which may be the **logit** function

$$g(\mu(X)) = \log\left(\frac{\mu(X)}{1 - \mu(X)}\right) = X^T\beta \tag{158}$$

or the **probit** function

$$g(\mu(X)) = \Phi^{-1}\big(\mu(X)\big) = X^T\beta \tag{159}$$

where Φ is the CDF of a standard Gaussian. The two functions can be seen in Figure 8.

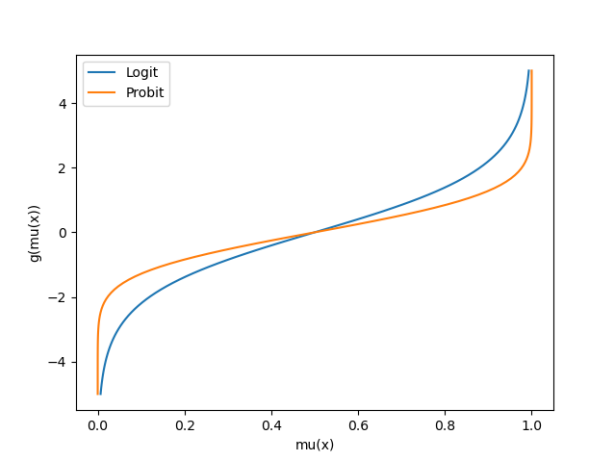


Figure 8: Logit and Probit Functions

Now there are many choices of functions we can take. In fact, if μ lives in $(0,1)$, then we can really just take our favorite distribution that has a density that is supported everywhere in $\mathbb{R}$ and take the inverse cdf as our link. So far, we have no reason to prefer one function to another, but in the next section, we will see that there are more natural choices.

7.3.1 Canonical Link Functions

Now let's summarize what we have. We assume that the conditional distribution $Y \mid X = x$ follows a distribution in the exponential family, which we can completely characterize by the cumulant generating function ψ. For different values of x, the conditional distribution will be paramaterized by different $\eta(x)$, and the resulting distribution P_η will have some mean $\mu(x)$, which is usually not the natural parameter η. Now, let's forget about our knowledge that $\psi'(\eta) = \mu$, but we know that there is some relationship between $\eta \leftrightarrow \mu$.

Given an x, I need to use the linear predictor $x^T\beta$ to predict $\mu(x)$, which can be done through the link function g.

$$g\big(\mu(x)\big) = x^T\beta \tag{160}$$

Now what would be a natural way of choosing this g? Note that our natural parameter η for this canonical family takes value on the entire real line. I must construct a function g that maps μ onto the entire real line, and so why not make it map to η. Therefore, we have

$$\eta(x) = g\big(\mu(x)\big) = x^T\beta \tag{161}$$

Definition 7.4 (Canonical Link)

The function g that links the mean μ to the canonical parameter θ is called the **canonical link**.

$$g(\mu) = \theta \tag{162}$$

Now using our knowledge that $\psi'(\eta) = \mu$, we can see that

$$g = (\psi')^{-1} \tag{163}$$

This is indeed a valid link function.

1. $\psi'' > 0$ since it models the variance, and so ψ' is strictly increasing and so $g = (\psi')^{-1}$ is also strictly increasing.
2. The domain of ψ' is the real line since it takes in the natural parameter η which exists over $\mathbb{R}$, so $\text{Im}(g) = \mathbb{R}$.

So, given our cumulant generating function ψ and our link function g, both satisfying

$$\psi'(\eta) = \mu \text{ and } g(\mu) = x^T\beta \tag{164}$$

we can combine them to get

$$(g \circ \psi')(\eta) = g(\mu) = x^T\beta \tag{165}$$

and so, even though the mean of the response variable is not linear with respect to x, the value of $(g \circ \psi')(\eta)$ is indeed linear. In fact, if we choose the canonical link, then the equation

$$\eta = x^T\beta \tag{166}$$

means that the natural parameter of our conditional distribution in the exponential family is linear with respect to x! From this we can find the conditional mean $\mu(x)$.

The reason we focus on canonical link functions is because, when the canonical link is used, the components of the model (the parameters of the linear predictor) have an additive effect on the response variable in the

transformed (linked) scale, which makes the interpretation of the results easier. It's also worth noting that while using the canonical link function has some desirable properties, it is not always the best or only choice, and other link functions may be used if they provide a better fit for the data or make more sense in the context of the problem at hand.

Let us evaluate some canonical link functions.

Example 7.14 (The Bernoulli in Canonical Form)

The Bernoulli has the canonical exponential form of

$$p_\eta(x) = \exp\left[x\eta - \log(1+e^\eta)\right] \tag{167}$$

where $\eta = \log\left(\frac{\theta}{1-\theta}\right)$. Since we have prior knowledge that $\theta = \mu$ (i.e. the expectation of a Bernoulli is the hyperparameter θ itself), we have a function that maps $\mu \mapsto \eta$.

$$\eta = g(\mu) = \log\left(\frac{\mu}{1-\mu}\right) \tag{168}$$

which gives us our result. We can also take the inverse of $\psi' = \frac{e^\eta}{1+e^\eta}$ to get our result

$$g(\mu) = (\psi')^{-1}(\mu) = \log\left(\frac{\mu}{1-\mu}\right) \tag{169}$$

7.4 Likelihood Optimization

Now let us have a bunch of data points $\{(x_n, y_n)\}_{n=1}^N$. By our model assumption, we know that the conditional distribution $Y \mid X = x_n$ is now of an exponential family with parameter $\eta_n = \eta(x_n)$ and density

$$p_{\eta_n}(y_n) = \exp\left[y_n\eta_n - \psi(\eta_n)\right]h(y_n) \tag{170}$$

Now we want to do likelihood optimization on β (not η or μ), and to do this, we must rewrite the density function in a way so that it depends on β. Given a link function g, note the following relationship between β and η:

$$\begin{aligned}\eta_n = \eta(x_n) &= (\psi')^{-1}(\mu(x_n)) \\ &= (\psi')^{-1}\big(g^{-1}(x_n^T\beta)\big) \\ &= h(x_n^T\beta)\end{aligned}$$

where for shorthand notation, we define $h \coloneqq (g \circ \psi')^{-1}$. Subtituting this into the above likelihood, taking the product of all N samples, and logarithming the equation gives us the following log likelihood to optimize over β.

$$\ell(\beta) = \log \prod_{n=1}^N p_{\eta_n}(y_n) = \sum_{n=1}^N y_n h(x_n^T\beta) - \psi(h(x_n^T\beta)) \tag{171}$$

where we dropped the $h(y_n)$ term at the end since it is a constant and does not matter. If g was the canonical link, then h is the identity, and we should have a linear relationship between $\eta(x_n) = x_n^T\beta$. This means that the η_n reduces only to $x_n^T\beta$, which is much more simple to optimize.

$$\ell(\beta) = \log \prod_{n=1}^N p_{\eta_n}(y_n) = \sum_{n=1}^N y_n x_n^T\beta - \psi(x_n^T\beta) \tag{172}$$

Note that the first term is linear w.r.t β, and ψ is convex, so the entire sum must be concave w.r.t. β. With this, we can bring in some tools of convex optimization to solve.

References

[MP43] Warren S. McCulloch and Walter Pitts. A logical calculus of the ideas immanent in nervous activity. *The bulletin of mathematical biophysics*, 5(4):115–133, 1943.